

Module 2: Intelligent Data Driven Applications on the Cloud

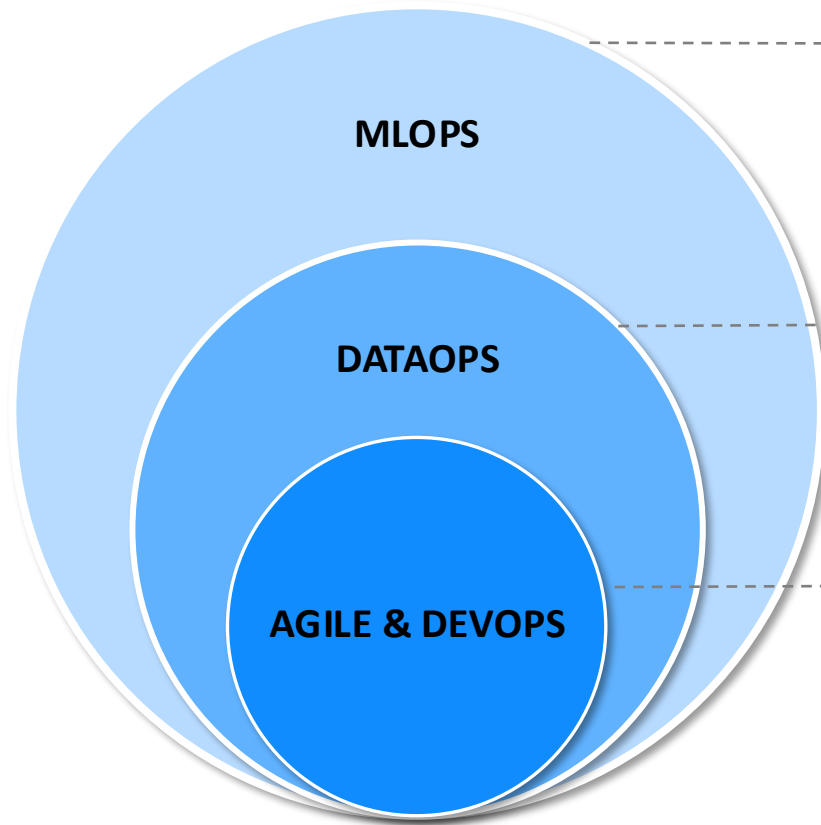
From notebooks to production-ready, AI-enabled applications

INTELLIGENT DATA DEVELOPMENT, JUNE 2026

Part 1: Data-driven Intelligence through DataOps & MLOps

INTELLIGENT DATA DEVELOPMENT, JUNE 2026

CONCEPTS COVERED – ZOOM IN



Agile = how we organize work
DevOps = how we deploy software
DataOps = how we deploy data pipelines
MLOps = how we deploy ML models

“MLOps or ML Ops is a **set of practices that aims to deploy and maintain machine learning models in production reliably and efficiently**. The word is a compound of "machine learning" and the continuous development practice of DevOps in the software field.”

WIKIPEDIA

“DataOps is a set of **practices, processes and technologies** that combines an **integrated and process-oriented perspective on data with automation and methods from agile software engineering** to improve quality, speed, and collaboration and promote a culture of continuous improvement in the area of data analytics.”

WIKIPEDIA

DevOps is a set of **practices** that works to **automate and integrate the processes between software development and IT teams**, so they can build, test, and release software faster and more reliably”

“Agile is an **iterative approach to project management and software development that helps teams deliver value to their customers faster and with fewer headaches**. Instead of betting everything on a "big bang" launch, an agile team delivers work in small, but consumable, increments.

ATLASSIAN

AGILE PRODUCT DELIVERY LIMITATIONS

Agile helps teams **build the right solution iteratively**, but without DevOps, they often **struggle to deliver** and operate it reliably **at scale**.

Agile Alone Isn't Enough – Why DevOps Matters

- Agile drives **collaboration** and **iteration** but lacks tools for **deployment, testing, and monitoring**
- DevOps ensures **reliable delivery** and **operational continuity**
- Together, they enable **fast, scalable, and production-ready** data science

Without DevOps, teams face:

- “It worked in dev, not in prod”
- Manual deployment errors
- No version control for models or data
- No monitoring for drift or failure

WHY DEVOPS MATTERS IN DATA SCIENCE

You built a forecasting model.

- Forecast Accuracy = 85%
- The business validates it.
- The model is approved.

But can the business actually use it?

WHY DEVOPS MATTERS IN DATA SCIENCE

You built a forecasting model.

- Forecast Accuracy = 85%
- The business validates it.
- The model is approved.

But can the business actually use it?

What needs to happen before the business can use it?

- Forecasts must run automatically every week
- Data must arrive on time
- Code changes must be tested before deployment
- Results must be delivered to planners and business users
- Forecast quality must be monitored over time
- The solution must be reproducible and maintainable

What can go wrong?

1. Missing Data

The forecast is scheduled to run Sunday night.

The shipments file does not arrive.

Q: Does the forecast still run?

2. Code Change

A Data Scientist updates the feature engineering logic.

Forecast accuracy drops from 85% to 60%.

Q: How do we identify which change caused the issue?

3. Knowledge Transfer

The original Data Scientist leaves the company.

Q: Can another person reproduce the model and maintain it?

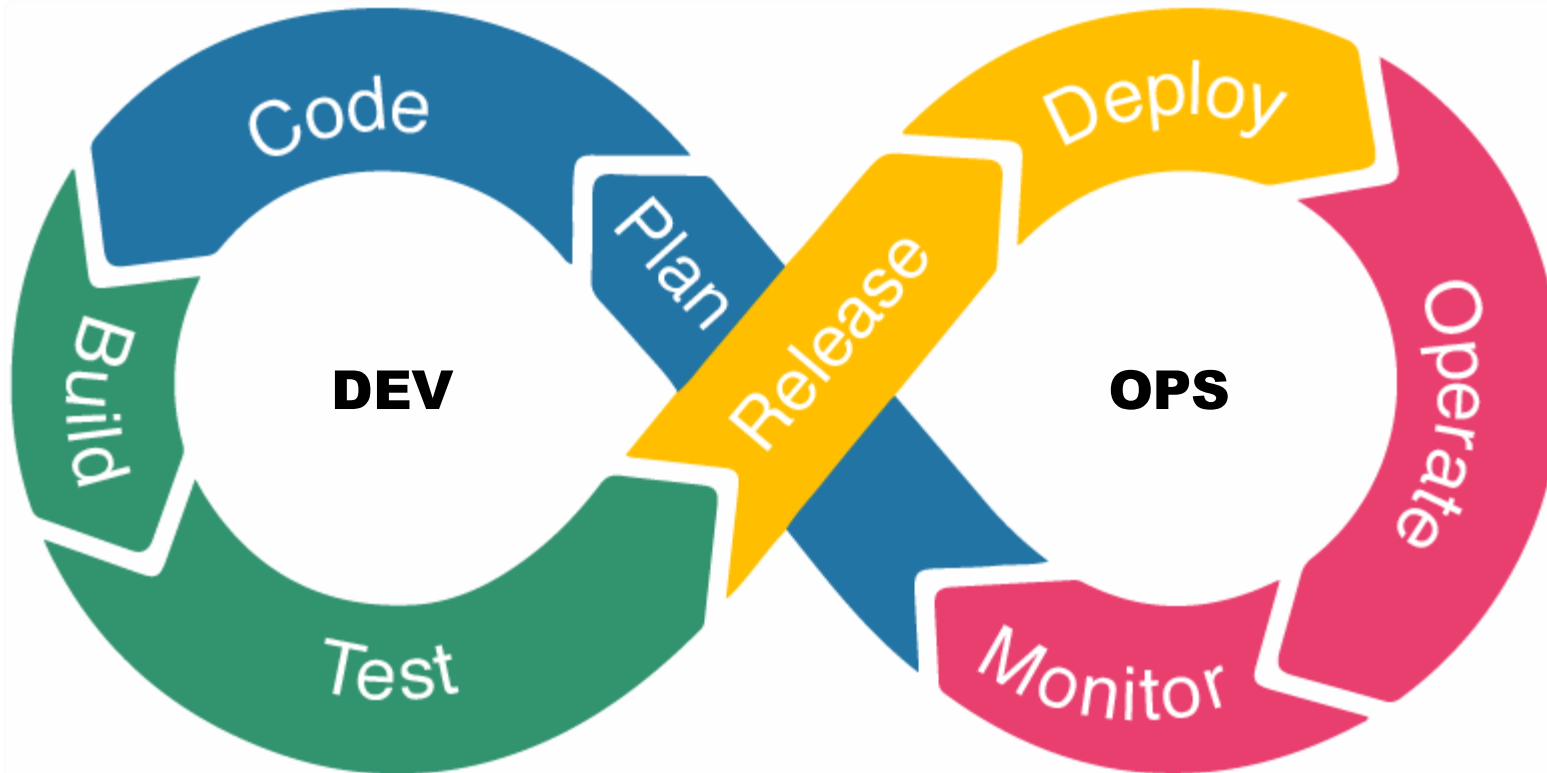
4. Business Audit

The business asks:

"Why did we forecast 1M units for this product three months ago?"

Q: Can we reproduce exactly the forecast generated at that point in time?

DEVOPS LIFECYCLE



- DevOps (Development + Operations) is a practice that allows a **single team to manage the entire application development life cycle**, that is, development, testing, deployment, and operations.
- It provides a **standardized structure to integrate DevOps practices into your delivery pipeline to increase the speed and reliability of the delivery process**. Meanwhile, it offers the required flexibility to meet changing demands.

DEVOPS LIFECYCLE

CONTINUOUS DEVELOPMENT PLAN & CODE

- **Goal:**
 - This phase involves the **planning** and **coding of the software**.
 - Product vision is decided. And developers begin developing the code for the application.
- **Tools:**
 - There are no DevOps tools that are required for planning
 - Jira, GitLab, Azure DevOps are usually tools used for maintaining the code

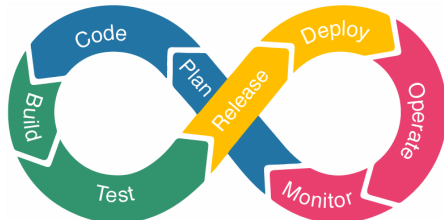
CONTINUOUS INTEGRATION BUILD

- **Goal:**
 - This stage is the **heart of the entire DevOps lifecycle**.
 - Developers require to commit changes to the source code more frequently (daily or weekly basis).
 - Then every commit is built, and this allows early detection of problems if they are present. **Building code is not only involved compilation**, but it also includes **unit testing, integration testing, code review, and packaging**.
- **Tools:**
 - Jenkins, Azure DevOps

TESTING

- **Goal:**
 - One of the longest stages of the DevOps lifecycle, where the **product is tested for bugs and/or errors**.
 - Tests can be automated through automation testing (saves a lot of time and effort for executing the tests instead of doing this manually)
 - After testing, the code is continuously integrated with the existing code.
- **Tools:**
 - Selenium, Jira, Azure DevOps

RELEASE



- **Goal:**
 - Point in time where the **build is ready for deployment into the production environment**.
 - By this stage, the application code has passed a series of manual and automated tests, and the Ops team is confident that breaking issues and regressions are unlikely.
 - Can be done automatically or in a regular release schedule. A manual approval process at the stage gate might be needed.
- **Tools:**
 - Jenkins, Azure DevOps

DEVOPS LIFECYCLE

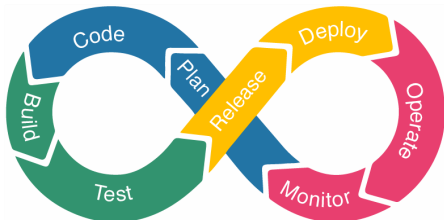
CONTINUOUS DEPLOYMENT

- **Goal:**
 - **Point in time when the code is deployment into the production environment.**
 - Code is deployed continuously, and **configuration management tools** are used to configure, manage and deploy various applications and services.
 - **Containerization** tools are also playing an essential role in the deployment phase. I.e. **Docker** is a popular tools that is used for this purpose. These tools help to produce consistency across development, staging, testing, and production environments.
- **Tools:**
 - Chef, Puppet, Ansible, and SaltStack.

OPERATE

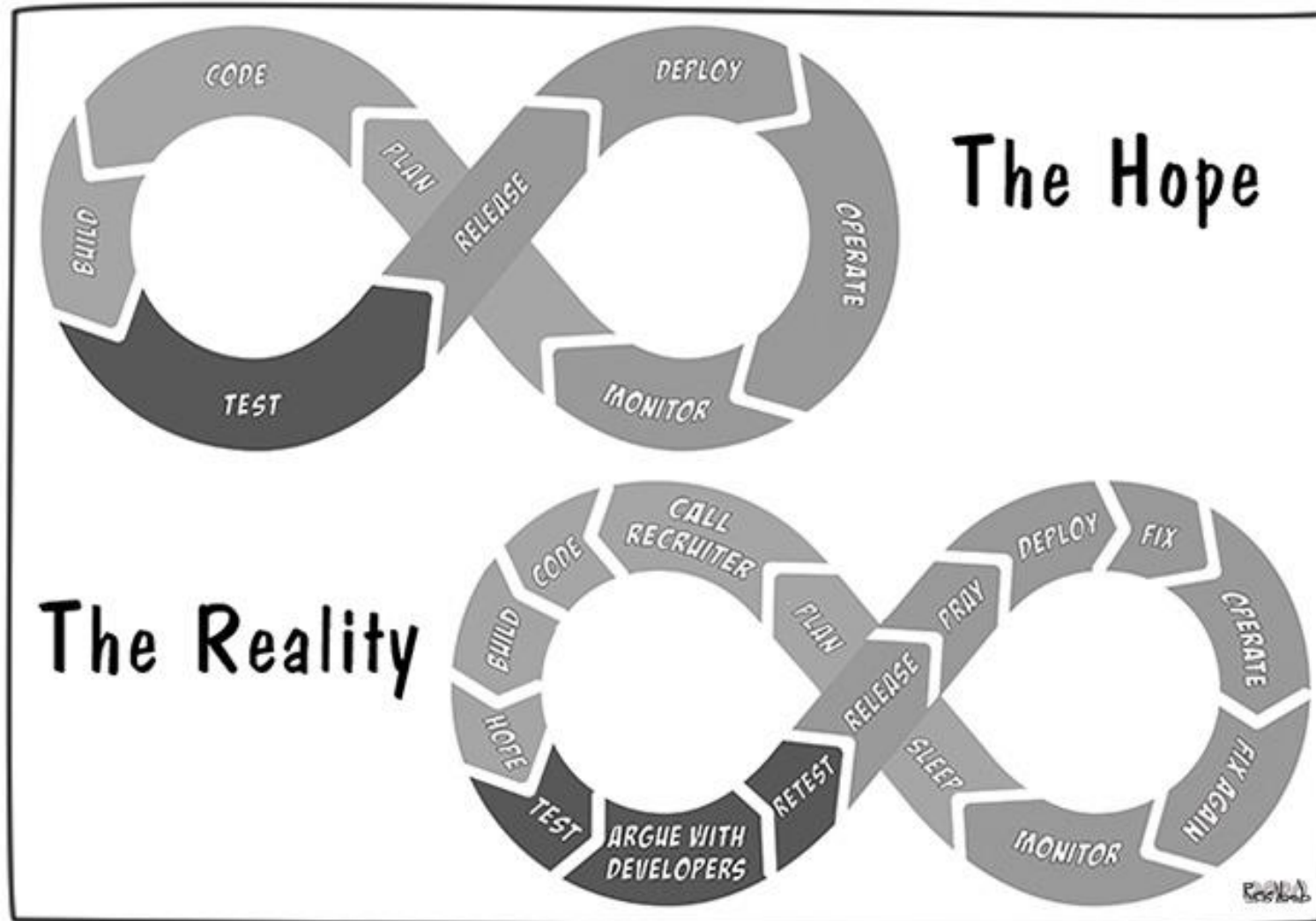
- **Goal:**
 - The new release is now live and being used by the customers.
 - **Operations rely heavily on monitoring to understand the health of the application.** It ensures that the application functions nominally and reliably throughout its operational life.
 - The **responsibility** of maintaining the application is primarily associated with **the operations team**. It can involve other teams in case of bug fixes.
 - Might be that a feedback loop is enabled to ensure customer inputs are considered.
- **Tools:**
 - DataDog, Splunk, Slack

MONITORING



- **Goal:**
 - Phase in which important information about the use of the software is recorded and carefully processed to find out trends and identify bottlenecks.
 - Continuous monitoring is the optimal way to understand both the application and the underlying environment and **detect anomalies quickly**. Monitoring can include every aspect of the application, from simple CPU and memory utilization to monitoring API calls.
- **Tools:**
 - Splunk, DataDog, any custom-made visualization tools

DEVOPS LIFECYCLE - REALITY



DEVOPS TOOLS FOR DS

Function

Version control

CI/CD

Data pipelines

Model tracking

Monitoring

Deployment

Tools

Git, DVC

GitHub Actions, GitLab CI, Azure DevOps

Airflow, Prefect

MLflow, Weights & Biases

Evidently AI, Prometheus

Docker, Kubernetes, FastAPI

Starter tip: Learn Git + MLflow + Airflow → foundational stack

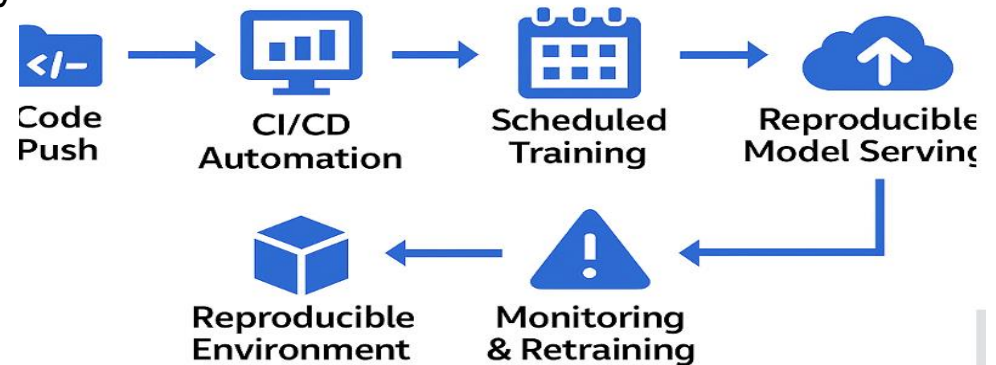
DEVOPS EXAMPLE: FORECASTING ENGINE

From Notebook to Production

Imagine you've built a machine learning model to forecast weekly

The business now expects:

- Forecasts available every Monday morning
- Monthly model retraining
- Results delivered automatically to planners
- Alerts when something goes wrong



How do we make sure the forecast continues to run reliably every week, even when code, data and people change?

How DevOps Solves the challenges:

- **Code Push:** Every change is versioned. If somebody breaks the forecast, we know exactly what changed
- **CI/CD Automation:** Before anything reaches production, automated tests verify that the code still works.
- **Scheduled Model Training:** The forecast runs automatically. Nobody needs to remember every Sunday night to launch it.
- **Reproducible Model Serving:** Forecasts must be available to the business through dashboards, APIs or planning systems.
- **Monitoring & Retraining:** If forecast accuracy drops or data quality deteriorates, alerts are generated and retraining can be triggered.
- **Reproducible Environments:** If another Data Scientist joins tomorrow, they should be able to reproduce exactly the same environment and results

DEVOPS WORKING SESSION – 20'

Goal: Your Forecasting Model Is Going Live

Time: 10 minutes

Work in small teams -> 10 mins to share ideas

Scenario:

You have built a forecasting model with **85% accuracy**. The business now wants forecasts delivered automatically every Monday morning. Your team is responsible for turning this notebook into a production-ready solution.

Your Task - Split into 5 groups, each group will discuss one question and prepare a 2-minute answer.

- **Group 1:** What must happen before the model can be used by the business?
- **Group 2:** What is the biggest risk if DevOps is not in place?
- **Group 3:** What would you automate first and why?
- **Group 4:** How would you know if the forecast failed or degraded?
- **Group 5:** How would another Data Scientist maintain and reproduce your solution?

DEVOPS WORKING SESSION – PROPOSED SOLUTION

Group	Question	Answer
Group 1	What must happen before the model can be used by the business?	The notebook must become a production pipeline : code in Git, tested, deployed to a controlled environment, scheduled to run weekly, connected to data sources, and outputs delivered automatically to planners.
Group 2	What is the biggest risk if DevOps is not in place?	The model may work once but fail repeatedly in production : manual runs, untested code changes, inconsistent environments, no rollback, no traceability , and no clear owner when something breaks .
Group 3	What would you automate first and why?	Automate testing and scheduled execution first. Testing prevents broken code from reaching production; scheduling ensures forecasts run every week without depending on a person manually launching notebooks.
Group 4	How would you know if the forecast failed or degraded?	Add monitoring and alerts : pipeline success/failure, data freshness, missing outputs, forecast accuracy, bias, drift, runtime errors, and abnormal forecast values.
Group 5	How would another Data Scientist maintain and reproduce your solution?	Use version control, documentation, reproducible environments, configuration files, model/data versioning, and clear run instructions . Another DS should know what code, data, parameters, and model version produced each forecast.

DATAOPS



DATAOPS - DEFINITION

WHAT IS DATAOPS?

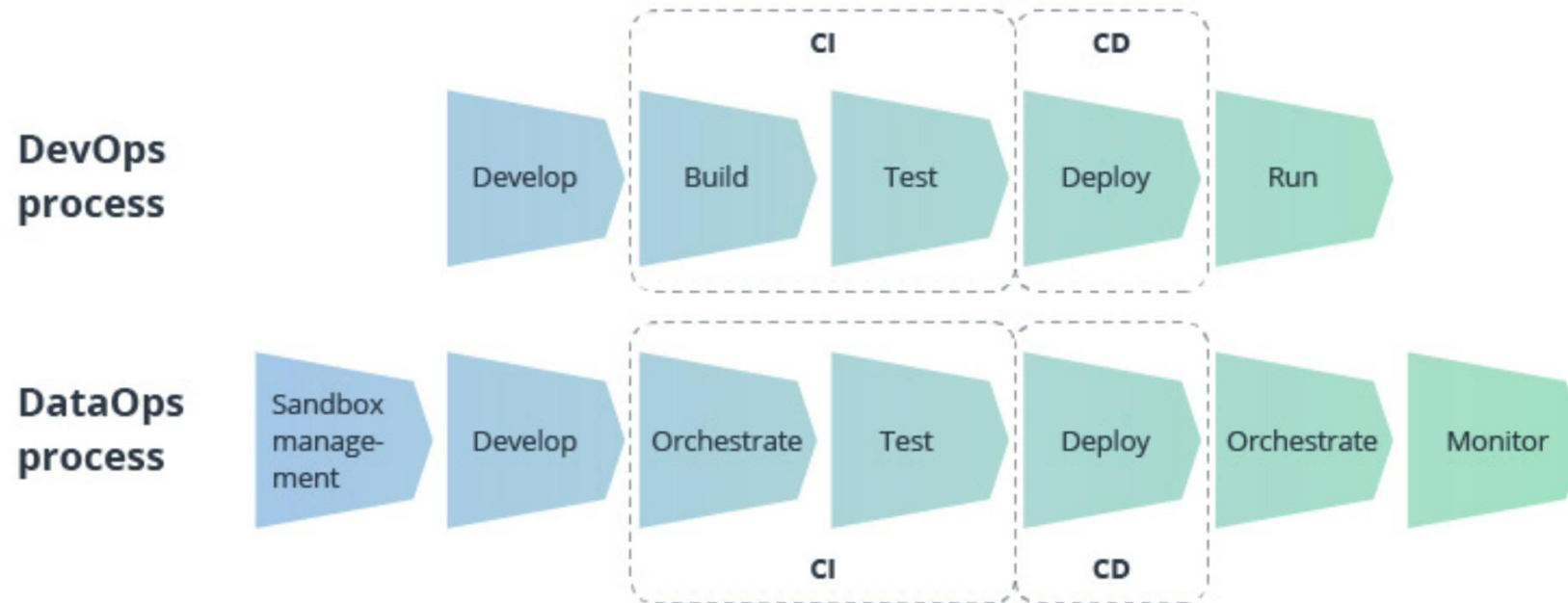
“DataOps (data operations) refers to **practices** that bring **speed and agility** to **end-to-end data pipelines** process, from collection to delivery.”

IBM

DATAOPS – THE NEW DEVOPS FOR A DATA DRIVEN ORGANIZATION

DataOps: The New DevOps for a Data-Driven World

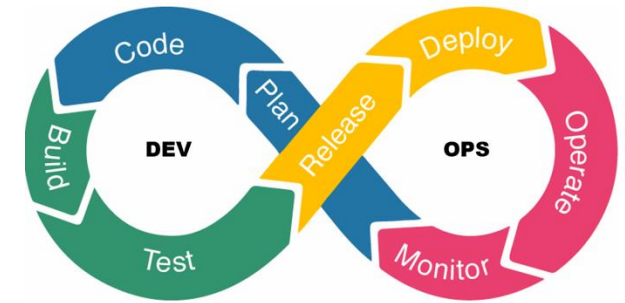
With data being today's most valuable commodity, companies are embracing DataOps as an integral part of their business



DATAOPS = DEVOPS for DATA

- Both rely on **Big Data** and the **Cloud**
- Both use advanced **process automation** for testing, data orchestration, deployment, and continuous monitoring.
- Both lean on **continuous integration** and **continuous deployment/delivery**.
- Both aim to **shorten the duration of release cycles** for software products and valuable datasets by following key **Agile principles**.
- Both target to achieve **faster time to market**, and **release high-quality products**
- The users of DataOps are data teams (data engineers, architects, analysts, scientists, and operations)

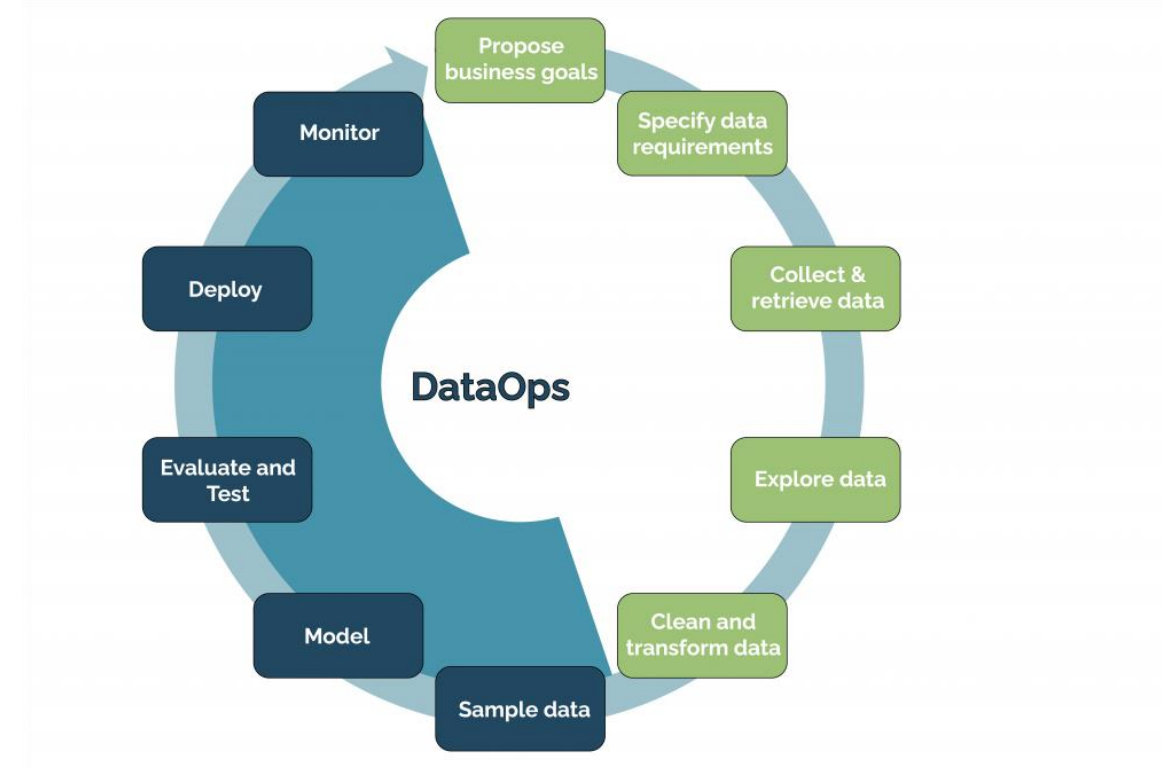
DATAOPS LIFECYCLE



- **Development**
 - In DataOps, the step may involve **building a new pipeline, changing a data model or redesigning a dashboard**.
 - **Example:** We add a new weather/promotions pipeline.
- **Testing**
 - Usually testing includes **testing of inputs, outputs and business logic**
 - These tests should be performed at each stage of the data analytics pipeline, to ensure that results of the particular data job meet expectations.
 - **Example:** We validate weather structure, schema, contents, data logic.
- **Deployment**
 - This presumes **moving data jobs between environments**, pushing them to the next stage, or deploying the entire pipeline in production.
 - Many times, given that Production data is not allowed in other environments than PROD, it may be the case that data is pushed in each environment separately.
 - **Example:** We move pipeline into the production environment, by passing from DEV to QA to PREPROD and finally to PROD.
- **Monitoring**
 - A must have to ensure data quality, monitoring allows data professionals to **identify bottlenecks, catch abnormal patterns, and measure adoption of changes**.
 - Monitoring often relies on AI-driven anomaly detection algorithms.
 - **Example:** We check data arrives when scheduled and with the right data quality checks in place.
- **Orchestration**
 - In the data world, **orchestration automates moving data between different stages**, monitoring progress, triggering autoscaling, and other operations related to the management of data flows.
 - It covers each stage and the end-to-end data analytics pipeline.
 - **Example:** Your choice orchestrator (Airflow or Azure Data Factory or AWS Step Functions) triggers everything.

DATAOPS – THE NEW DEVOPS FOR A DATA DRIVEN ORGANIZATION

- **Accelerates** the ability of data-analytics teams to create and publish new **analytics solutions to end users**, while still being faster, cheaper and higher quality
- **Reduces time** to insight
- **Improves analytic quality**
- **Lowers the marginal cost** to ask the next business question
- Improves analytic **team morale** by going beyond hope, heroism and caution
- Promotes **team efficiency** through agile process, reuse and refactoring



DATAOPS WORKING SESSION – 35'

Goal: Your Forecasting Model Is Going Live

Time: 15 minutes

Work in small teams -> 20 mins to share ideas

Scenario:

Monday, 4am. The forecasting engine is about to run. Several data issues have been detected. Your team is responsible for deciding what to do.

Your Task - Split into 5 groups, each group will discuss one question and prepare a 2-minute answer.

- **Group 1: Weather data is missing.** Should the forecast run? What checks should be in place? Who should be alerted?
- **Group 2: Promotions data contain 0 rows.** How do you detect the issue? Should the forecast continue? What is the business impact?
- **Group 3: The Shipments file arrived, but a column name changed.** The pipeline fails. How can DataOps prevent this? What controls would you implement?
- **Group 4: The data usually arrives at 2 AM. Today it arrives at 6 AM. The forecast is scheduled for 4 AM.**

What should happen? Should the forecast wait? Should it run? How would you automate the decision?

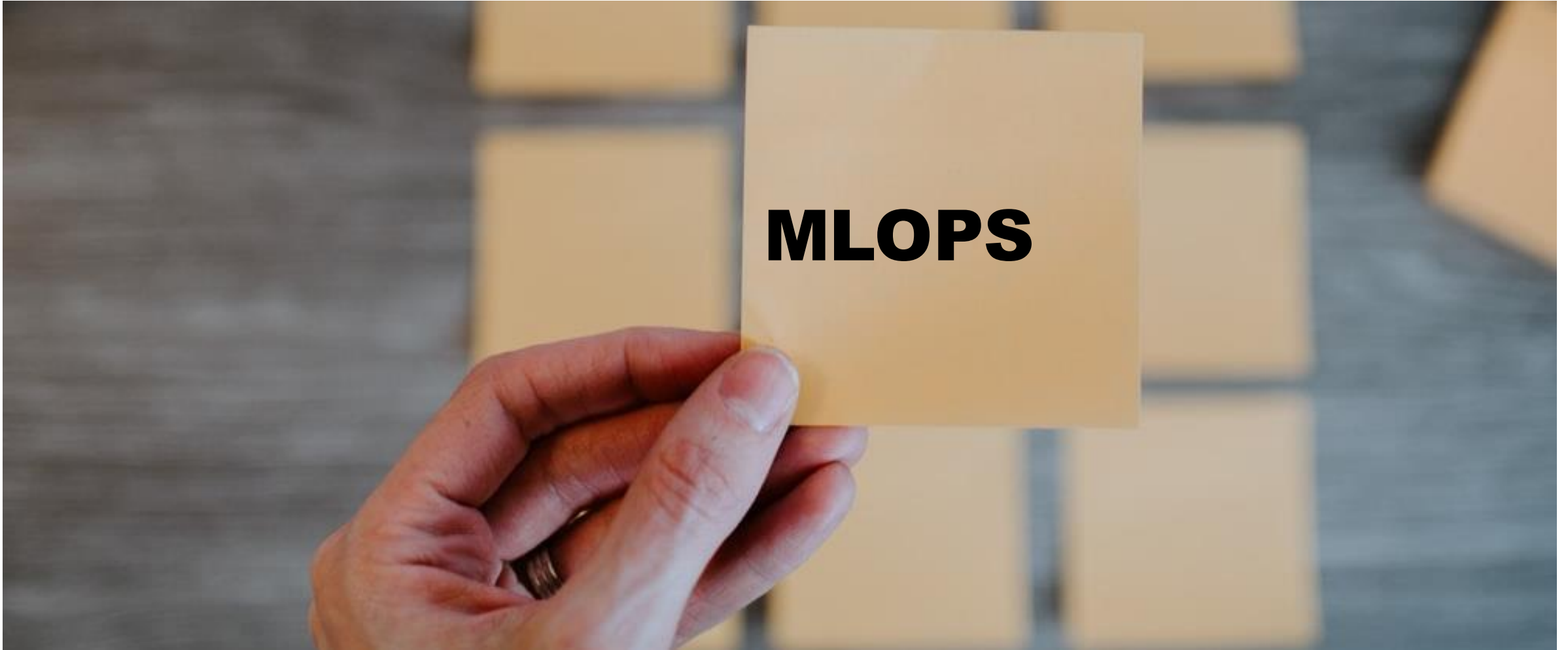
- **Group 5:** The data arrives successfully, however, volumes suddenly increase by 300%.

No errors are reported. How would you detect this? How would you distinguish a true business event from a data quality issue?

DATAOPS WORKING SESSION – PROPOSED SOLUTION

Group	Incident	Proposed Solution
Group 1	Missing Data — Weather dataset did not arrive	Add data freshness checks and dependency validation before the forecast runs. If weather is mandatory, stop the run and alert Data Engineering / Forecasting Support. If it is optional, use an approved fallback such as last available weather or historical averages.
Group 2	Empty Dataset — Promotions table contains 0 rows	Add row count validation and business rules. If promotions are expected for the market/week, the pipeline should fail fast, block forecast publication, and alert the data owner.
Group 3	Schema Change — Shipments file arrived but a column name changed	Implement schema validation and data contracts. The pipeline should check column names, data types, mandatory fields, and expected formats before transformation starts.
Group 4	Late Data Arrival — Data usually arrives at 2 AM but arrives at 6 AM	Use dependency-based orchestration instead of only time-based scheduling. The forecast should wait until required data is ready, with a defined timeout and escalation process.
Group 5	Suspicious Data — Volumes suddenly increase by 300%	Add anomaly detection and volume threshold checks comparing current data with historical patterns. If the increase is abnormal, trigger manual review before allowing the forecast to continue.

MLOPS



MLOPS - DEFINITION

WHAT IS MLOPS?

“MLOps or ML Ops is a set of practices that aims to deploy and maintain machine learning models in production reliably and efficiently. The word is a compound of "machine learning" and the continuous development practice of DevOps in the software field.”

WIKIPEDIA

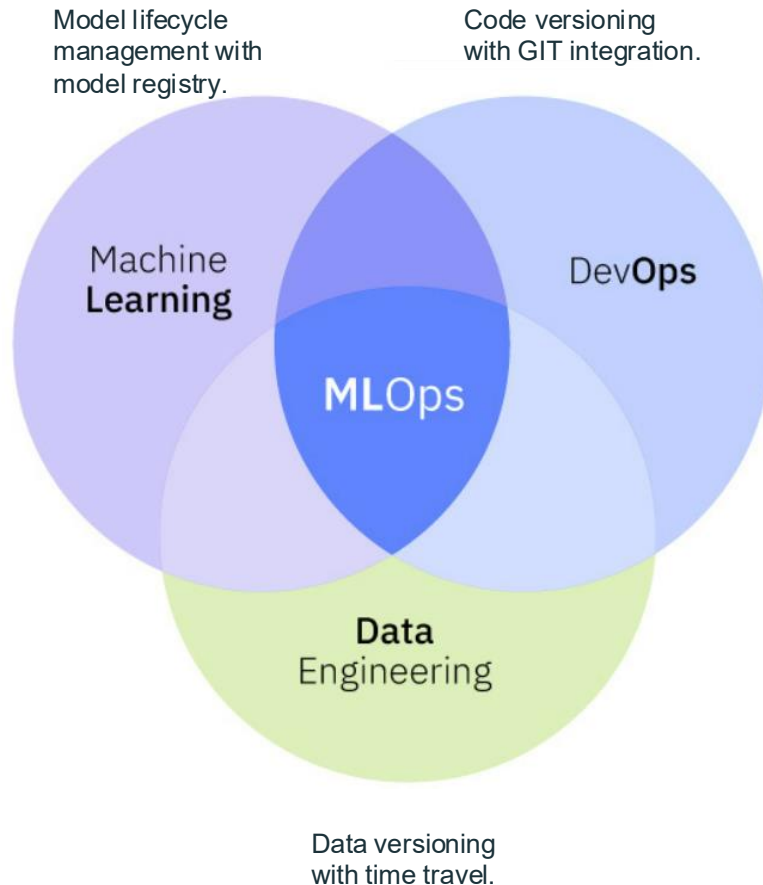
MLOPS – REALITY

87% of Data Science projects never make it to Production

MLOPS – MATURITY ACROSS ORGANIZATIONS

LEVEL	DESCRIPTION	HIGHLIGHTS	TECHNOLOGY
0	No MLOps	• Difficult to manage full machine learning model lifecycle • The teams are siloed, and releases are painful • Most systems exist as "black boxes," little feedback during/post deployment	• Manual builds and deployments • Manual testing of model and application • No centralized tracking of model performance • Training of model is manual
1	DevOps but no MLOps	• Releases are less painful than No MLOps, but rely on Data Team for every new model • Still limited feedback on how well a model performs in production • Difficult to trace/reproduce results	• Automated builds • Automated tests for application code
2	Automated Training	• Training environment is fully managed and traceable • Easy to reproduce model • Releases are manual, but low friction	• Automated model training • Centralized tracking of model training performance • Model management
3	Automated Model Deployment	• Releases are low friction and automatic • Full traceability from deployment back to original data • Entire environment managed: train > test > production	• Integrated A/B testing of model performance for deployment • Automated tests for all code • Centralized training of model training performance
4	Full MLOps Automated Operations	• Full system automated and easily monitored • Production systems are providing information on how to improve and, in some cases, automatically improve with new models • Approaching a zero-downtime system	• Automated model training and testing • Verbose, centralized metrics from deployed model

MLOPS – ML + DATOPS + DEVOPS



- In classical software development, **a version of the code produces a version of the software.**
- The introduction of data changes how code works in the development process but also that data is another aspect that can change.
- Data as a new variable in the development process **increases complexity drastically**, as to reproduce results you need to be able to reproduce the data that you used.
- The matter of **versioning both data and code** becomes increasingly complex when you start thinking about whole machine learning systems which have steps for data preparation, augmentation and generation.
- **In MLOps a version of the code and a version of the data together produce a version of the ML model.**

MLOPS – PERSONAS

DATA SCIENTIST

- Tasked with finding data-driven solutions to business problems.
- Tasks: data wrangling, statistical analysis, data visualization and applying machine learning. While the primary purpose of data science is to **explore data and build models**, often a large portion of time is spent manipulating data to suit the use case.

DATA ENGINEER

- Responsible for the infrastructure that ensures data is stored and available for data scientists.
- In charge of building the systems that **ingest raw data from various sources and store it in a centralized data lake**.
- Data engineers often also build other data warehouses that derive data from the data lake but transform it so that it is more

ML ENGINEER

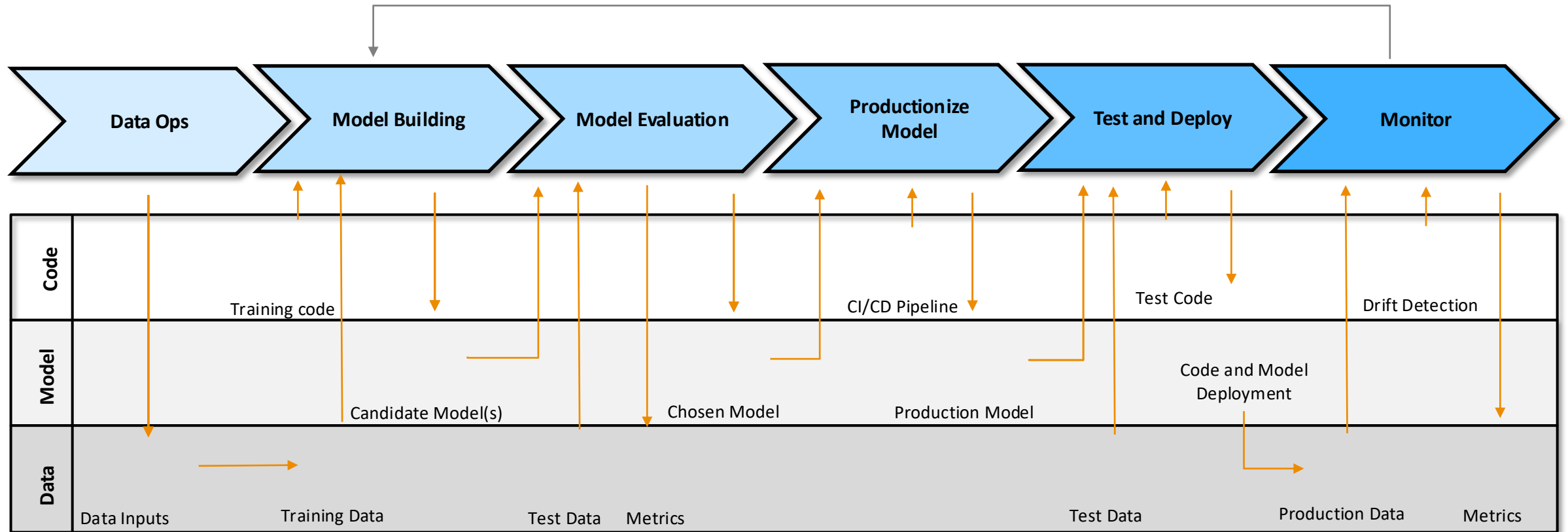
- Rather new role that is becoming increasingly relevant as companies push to get real-world value from machine learning. ML Engineers take the theoretical concepts explored by Data Scientists and build them into a production-scale system.
- Software engineering knowledge such as **continuous integration (CI) and continuous delivery (CD)** is essential.
- They tackle automating and scaling model training, testing models before deployment and deploying and monitoring models in production. Code rewrite and/or restructure code is a must.

DEVOPS ENGINEER

- Combine **software engineering** and **cloud infrastructure knowledge**.
- In machine learning projects, DevOps is the connection between the ML model and the end-user application..
- They are in charge to ensure the ML models deployed don't degrade the scalability, stability and responsiveness of the end-user application.

MLOPS – LIFECYCLE

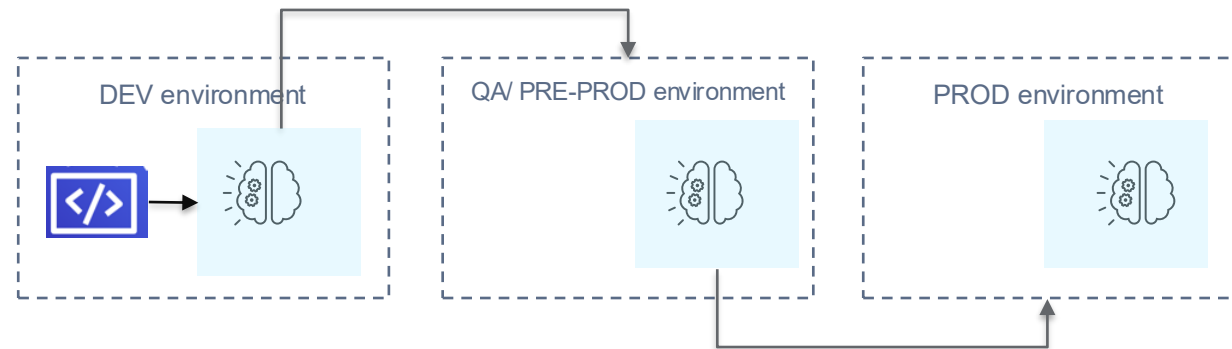
MLOps provides a framework that addresses the full life cycle of ML models including development, testing, deployment and monitoring that fulfills the needs of a classic CI/CD process and operation of the deployed machine learning system



MLOPS – DEPLOYMENT MODELS

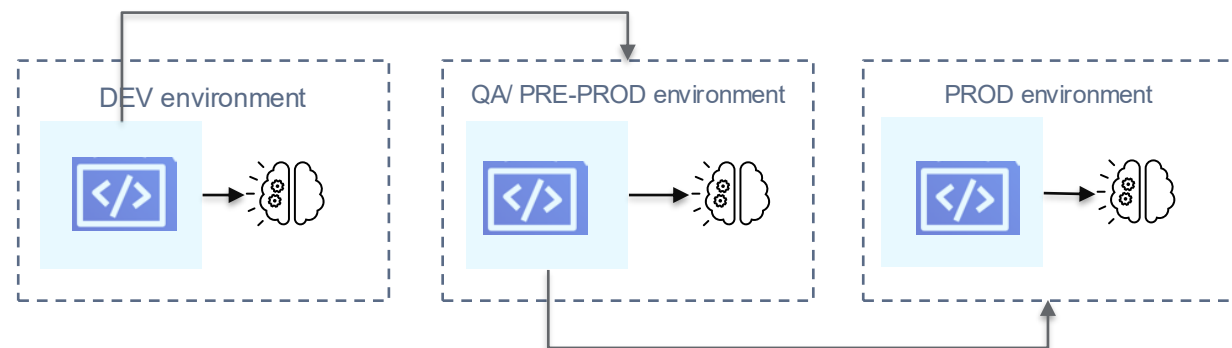
DEPLOY MODELS

- The model artifact is generated by training code in the development environment.
- The artifact is then pushed to the QA/ testing area, before finally being deployed into production.
- **Limitation:** In real life DEV environments can't use production data. Automated model retraining is not supported.



DEPLOY CODE

- Application code is developed in the DEV environment, and this code is moved to QA/testing and then production.
- Models are trained in each environment.
- Allows training on production data while respecting access controls.
- Safer to set up automated retraining since code goes through testing.



MLOPS WORKING SESSION – 20'

Goal: Your Forecasting Model Is Going Live

Time: 10 minutes

Work in small teams -> 20 mins to share ideas

Scenario: The forecasting engine has been running successfully in production for six months. All DataOps checks passed: data arrived on time, no missing files, no schema issues, no pipeline failures. However, forecast accuracy has dropped from **85% to 60%**. Your team is responsible for investigating the problem.

Your Task - Split into 5 groups, each group will discuss one question and prepare a 2-minute answer.

Group 1: Customer purchasing patterns have changed significantly compared to the data used for training

Questions: How would you detect this? What metrics would you monitor?

Group 2: Customer behavior has changed because of a market event, inflation, new competitors, or a major promotion strategy change

Questions: Why might the model no longer be valid? When should the model be retrained?

Group 3: A new feature engineering release was deployed one month ago

Questions: How would you determine whether the new release caused the issue? What versioning controls should be in place?

Group 4: The model continues to run successfully every week.

Questions: What KPIs would you monitor? What thresholds would trigger an alert?

Group 5: The model was trained six months ago

Questions: How often should retraining occur? Should retraining be time-based or performance-based?

MLOPS WORKING SESSION – PROPOSED SOLUTION

Group	Incident	Proposed Solution
Group 1	Data Drift – Customer purchasing patterns have changed significantly compared to the data used for training.	Monitor feature distributions and compare production data with training data. Use drift detection metrics and dashboards. If significant drift is detected, investigate the cause and consider retraining the model.
Group 2	Concept Drift – Customer behavior changed due to inflation, new competitors, or changes in promotion strategy.	Monitor forecast accuracy and bias over time. Even if input data looks similar, relationships between inputs and demand may have changed. Retrain, recalibrate, or redesign the model when performance deteriorates.
Group 3	Code & Feature Engineering Changes – A new feature engineering release was deployed one month ago.	Use Git versioning, model registries, experiment tracking, and CI/CD pipelines. Compare model performance before and after the release and rollback if the degradation is linked to the new code version.
Group 4	Monitoring & Alerting – The model continues to run successfully every week.	Monitor forecast accuracy, bias, drift metrics, runtime failures, missing predictions, and business KPIs. Define thresholds that automatically trigger alerts and investigations.
Group 5	Retraining Strategy – The model was trained six months ago.	Establish retraining policies based on time (e.g., monthly), performance thresholds (accuracy below target), or drift detection. Validate new models before deployment and keep the ability to rollback to previous versions.